

Unreal Engine Plugins et Tools

Table des matières

1	Introduction	1
2	Plugin	1
2.1	Définition	1
2.2	Contenu	1
2.2.1	Code	1
2.2.2	Assets	1
2.3	Plugins moteurs	1
2.4	Plugins projet	1
3	Création d'un Plugin	1
4	Plugin Génération Procédurale	1
4.1	Introduction	1
4.2	Architecture	2
4.3	Génération d'objets	2
4.3.1	Poisson Disk algorithm	2
4.3.2	Edition de spline	2
4.3.3	UI - Unreal Engine Slate	2
4.3.4	Génération adaptée	2
4.4	Optimisation du plugin	2
4.4.1	Multithreading	3
4.5	Exportation du plugin	3

1 Introduction

Ce rapport documentera la conception d'un plugin Editor de génération procédurale de props ainsi qu'aportera des explications sur les principaux sujets techniques qui y sont liés, afin de servir de référence sur laquelle je pourrais revenir au besoin.

2 Plugin

2.1 Définition

Un plugin est une collection de codes et outils externes pouvant être activés/désactivés facilement depuis l'éditeur Unreal. Ils consistent en des fonctionnalités gameplay ou des fonctionnalités moteurs permettant d'étendre les capacités d'UE.

2.2 Contenu

2.2.1 Code

Les plugins peuvent contenir du code bien que ce ne soit pas impératif, dans ce cas là on aura un dossier contenant le code source du plugin ainsi que les dossier Intermediate et binaries contenu séparément.

Les plugins peuvent être divisés par modules, et tout comme eux nécessitent d'être paramétrés par des fichiers Build.cs afin de gérer la compilation du code du plugin. Au global leur fonctionnement est quasiment identiques aux modules.

2.2.2 Assets

Ils peuvent également contenir les assets nécessaire à son fonctionnement.

2.3 Plugins moteurs

Les plugins moteurs peuvent être intégré à tous les projets, offrant des fonctionnalités de bases sans nécessiter de modifier le code moteur.

2.4 Plugins projet

Intégré au projet ils compilent avec, ils sont contenus dans un dossier Plugins à la racine.

3 Création d'un Plugin

Un Plugin peut facilement être créé depuis la fenêtre plugin d'Unreal, la seule chose à configurer sera le type de plugin, dans mon cas le Blank fait très bien l'affaire.

Pour que le Plugin soit compilable il est important comme dit plus tôt de correctement configurer le plugin ainsi que s'assurer de la bonne tenue des modules qui le composent.

4 Plugin Génération Procédurale

4.1 Introduction

Sur Unreal Engine il existe déjà un plugin très connu pour cette fonction : PCG, ceci dit il comporte certaines limites, il s'agit d'un outil permettant de créer ses propres systèmes de génération procédurale beaucoup plus simplement et avec une grande flexibilité mais ne permet pas un usage quasi instantané pour une personne qui ne maîtrise pas spécialement les outils de codes ou visual scripting du moteur de jeu. C'est pour cela que j'ai décidé de développer mon propre plugin de génération procédurale sans même reprendre la base de PCG, il y a aussi une volonté ici d'expérimenter et apprendre la création d'un plugin de a à z.

4.2 Architecture

Le module est composé de deux modules : Runtime et Editor, permettant un fonctionnement modulaire et une architecture solide. Coté Editor on pourra alors utiliser des outils comme Editor Utility Widget pour avoir un plugin prêt à l'emploi dans l'interface d'UE.

4.3 Génération d'objets

Dans un premier temps on utilise un actor, auquel on ajoutera un component permettant de générer des mesh autour de lui avec Instanced Mesh et d'autres paramètres, c'est un fonctionnement simple mais qui offre déjà une bonne base pour notre tool.

4.3.1 Poisson Disk algorithm

Pour déterminer les points sur lesquels nos mesh vont être instanciés selon les modalités que l'on souhaite pour notre plugin (espace minimale réglable entre les mesh, nombre de meshes paramétrable, surface sur laquelle les points apparaissent, etc) on utilisera un Poisson Disk algorithm.

Cette algorithm permet de générer des points à partir d'un premier, depuis celui ci on en crée un second selon un angle de 30 à 60 degrés à une distance R modulable et ainsi de suite, à chaque point créé on s'assure qu'il n'est pas trop proche d'un autre point voisin. Pour déterminer les points voisins on subdivise la zone en grille où chaque cellule a une dimension de $R/\sqrt{2}$, nous permettant d'être sûr qu'aucun point ne puisse partager la même cellule. Pour vérifier les voisins on boucle donc dans une aire de $5*5$ cellule autour de celui ci, si la recherche est concluante on ajoute définitivement le point.

4.3.2 Edition de spline

Le spline editing étant déjà géré par Unreal la principale difficulté à son propos viendra plus tard : récupérer ses données pour adapter la génération à elle. Pour le moment on crée juste un custom spline component qu'on peut manipuler dans la scène.

4.3.3 UI - Unreal Engine Slate

Slate est un module intégré à Unreal engine permettant de créer des interfaces utilisateur selon le même modèle que celle du moteur. Pour la mettre en place on crée une classe pour notre fenetre qui n'héritera d'aucune autre, dans notre cas on veut une UI dockable, pour ca on ajoute une fonction qui va créer la tab.

On appelle ensuite cette fonction dans le cpp du module concerné, en mode editor pour nous, afin qu'il ajoute l'UI à la section Windows du moteur à son lancement.

Une fois l'UI ajoutée on commence à paramétrer ses fonctionnalités, à l'intérieur d'une fonction dédiée englobant la tab on va ajouter différents slot, qui peuvent constituer en des boutons comme des valeurs twickables, on peut les définir à notre convenance avec des fonctions à disposition que je ne détaillerai pas.

4.3.4 Génération adaptée

Maintenant que l'on a des spline et une UI sur laquelle on peut paramétrer la génération il nous suffit juste de l'adapter à notre spline, pour cela on peut définir dans l'interface la spline sur laquelle on veut jouer.

Ensuite on va définir une bounding box autour de cette spline pour trier les points de generations valides déterminés plus tôt, suite à cela on précise davantage la vérification en supprimant tous les points en dehors de limites de la spline avec un algorithm de raycasting.

4.4 Optimisation du plugin

La génération causant des problèmes de performances évidents avec des freeze très longs nous essayerons d'explorer quelques méthodes d'optimisation.

4.4.1 Multithreading

Pour commencer on utilisera simplement le multithreading, permettant d'éviter le freeze en éditeur, cependant il est important de retenir que certains éléments de notre code doivent obligatoirement tourner sur le mainthread. Je pense notamment au Uobject dont l'initialisation et l'update se fait sur le game thread qui s'assure de la bonne stabilité de la boucle de gameplay.

Sur Unreal Engine on a accès à plusieurs méthodes de multithreading différente :

- Async (basique) : On appelle cette fonction lambda `Async(EAsyncExecution : :ThreadPool, [=]()` dans notre code, elle permet de déporter sa charge de travail sur un thread secondaire automatiquement sans bloquer le Game thread.

- Async Task : Cette option donne moins de controle mais permet d'exécuter facilement une tache sur le game thread ou un thread du task graph, il permet notamment de revenir au main thread apres un async.

- ParallelFor : Cette option permet d'itérer sur une liste en parallélisant l'exécution des itérations, en répartissant la charge de calcul sur les threads disponibles (via le Task Graph d'Unreal). Elle est particulièrement utile pour les boucles et calculs avec un grand nombre d'itérations indépendantes.

- NonAbandonableTask : Lance une tache qui se terminera forcément avant que le programme ne continue, quitte à bloquer le main thread.

- Future Promise : Structure de requete multithread permettant de lancer un clacul sur un thread secondaire et le recuperer ensuite sur le game thread.

4.5 Exportation du plugin

Pour exporter un plugin il suffit de retourner dans la fenêtre plugin, le trouver et le packager